

Informe de Seguridad — OWASP Top 10 (2021)

Sistema MediClick — Plataforma de gestión clínica multi-tenant

Proyecto	MediClick (NestJS + GraphQL + Prisma + PostgreSQL / Next.js)
Alcance	Auditoría y remediación dirigida sobre las 10 categorías OWASP, backend <code>server/</code>
Rama de trabajo	<code>feat/payments-mp-redirect</code>
Integrado a main	PR #35 — merge commit <code>e754494</code> (2026-06-30)
Fecha del informe	30 de junio de 2026

1. Resumen ejecutivo

Se realizó una auditoría dirigida del proyecto MediClick contra las 10 categorías del **OWASP Top 10 (edición 2021)**. De las categorías evaluadas, se identificaron y corrigieron hallazgos concretos en **A01 (Control de acceso roto)**, **A02 (Fallas criptográficas)**, **A06 (Componentes vulnerables)** y **A09 (Fallas de registro y monitoreo)**. El hallazgo más severo —ausencia de aislamiento multi-tenant en la API GraphQL, la única API autenticada del sistema— fue verificado a nivel de código y en tiempo de ejecución, y corregido con prueba de regresión. El resto de las categorías (A03, A04, A05, A07, A08, A10) no fueron objeto de auditoría dirigida en esta ronda; se documentan al final con las mitigaciones estructurales existentes y lo pendiente de revisar.

Categoría	Nombre	Estado	Resultado
A01	Control de acceso roto (Broken Access Control)	CORREGIDO	Tenant scoping GraphQL + guard defense-in-depth en updateClinic + auditoría read-before-write
A02	Fallas criptográficas (Cryptographic Failures)	CORREGIDO	Cost factor de bcrypt elevado de 10 a 12
A03	Inyección (Injection)	NO AUDITADO	Mitigación estructural por uso de Prisma ORM (queries parametrizadas) y class-validator; sin auditoría dirigida
A04	Diseño inseguro (Insecure Design)	NO AUDITADO	Fuera de alcance en esta ronda
A05	Configuración insegura (Security Misconfiguration)	NO AUDITADO	Fuera de alcance en esta ronda
A06	Componentes vulnerables (Vulnerable & Outdated Components)	PARCIAL	Auditoría automatizada agregada a CI (modo informativo); deuda preexistente sin sanear

A07	Fallas de identificación y autenticación	NO AUDITADO	JWT + Passport ya en uso; mejora de A02 aplica parcialmente; sin auditoría dirigida
A08	Fallas de integridad de software y datos	NO AUDITADO	Fuera de alcance en esta ronda
A09	Fallas de registro y monitoreo (Logging & Monitoring)	CORREGIDO	Bitácora de auditoría de seguridad append-only (login fallido + acceso denegado)
A10	Server-Side Request Forgery (SSRF)	NO AUDITADO	Fuera de alcance en esta ronda

2. A01 — Control de acceso roto (Broken Access Control)

2.1 Hallazgo principal: el aislamiento multi-tenant no se aplicaba en GraphQL

Severidad: Crítica

MediClick es multi-tenant: cada clínica es un tenant y los datos deben filtrarse por `clinicId`. El filtrado automático se implementa con una extensión de Prisma (`prisma.tenant`) que inyecta `clinicId` en las consultas, alimentada por un `TenantInterceptor` que lee el usuario autenticado del request.

El interceptor obtenía el request con `context.switchToHttp().getRequest()`. Esa llamada es válida en endpoints REST, pero en un *resolver* de GraphQL, `getArgByIndex(0)` retorna el **root del resolver**, no el request HTTP — por lo tanto `clinicId` quedaba `null` y la extensión de Prisma no aplicaba ningún filtro de tenant. Como GraphQL es la **única API autenticada** del sistema (REST solo expone el webhook público de MercadoPago), el mecanismo de aislamiento no protegía efectivamente ninguna consulta real del producto.

Verificación: se reprodujo el `ExecutionContext` real que NestJS construye para resolvers GraphQL (`args = [root, args, {req}, info]`) y se confirmó en tiempo de ejecución que `switchToHttp().getRequest()` devolvía `undefined` /objeto vacío bajo ese contexto.

Corrección: se reemplazó la obtención del request por un helper existente (`getRequestFromContext`) que distingue el tipo de contexto y usa `GqlExecutionContext.create(context).getContext().req` cuando corresponde a GraphQL.

```
// server/src/shared/interceptors/tenant.interceptor.ts
const req = getRequestFromContext(context) as { user?: { clinicId?: number | null } };
const clinicId: number | null = req?.user?.clinicId ?? null;
```

Mitigación adicional ya existente (defensa en profundidad): las mutaciones de escritura por id ya seguían, en su mayoría, el patrón *read-before-write* (leer el recurso antes de escribir), lo que acotaba el impacto del bug a IDOR de **lectura** en queries — la escritura estaba parcialmente protegida incluso antes del fix.

2.2 Auditoría de read-before-write en escrituras por id

Se revisaron las ~30 mutaciones que escriben por id (`update` / `delete` / `upsert` sobre clave primaria) para verificar que ninguna permita modificar un recurso de otro tenant (IDOR de escritura). Conclusión: **el invariante se cumple** en todos los casos, mediante una de tres vías:

- **Lectura ya filtrada por tenant** (`prisma.tenant.X.findFirst`) — appointments, doctors, medical-history, prescriptions, specialties.
- **Lectura + verificación explícita** de `clinicId` en el caso de uso, lanzando `ForbiddenException` si no coincide — availability, categories, holidays, schedule-blocks, roles, users.

- **Verificación de pertenencia (ownership)** contra el paciente autenticado del token — lista de espera (cancelar / aceptar oferta).

2.3 Hallazgo secundario: `updateClinic` sin verificación de tenant

Severidad: Baja (riesgo latente, no explotable hoy)

La mutación que actualiza una sede (`UpdateClinicUseCase`) no verificaba que el solicitante solo pudiera modificar **su propia** sede. No era explotable en el estado actual del sistema porque únicamente el permiso `MANAGE:ALL` (rol de plataforma) tiene `UPDATE:CLINICS` asignado, y para ese rol el acceso cruzado entre sedes es intencional. Aun así, se corrigió como defensa en profundidad: si en el futuro se crea un rol de administrador de clínica con permisos acotados, el control ya está en el caso de uso y no depende únicamente de la capa de permisos.

Por qué se discriminó por rol de sistema y no por `clinicId` : el usuario "admin de clínica" sembrado en el sistema usa el rol de plataforma `ADMIN` (equivalente a `MANAGE:ALL`) con un `clinicId` no nulo. Un guard ingenuo del tipo `if (clinicId && id !== clinicId)` habría bloqueado a ese super-administrador al editar una sede distinta a la suya. La solución correcta discrimina por el **rol de sistema** del solicitante (`SUPER_ADMIN` / `ADMIN` pueden administrar cualquier sede; el resto de roles, solo la propia).

```
// server/src/modules/clinics/application/use-cases/update-clinic.use-case.ts
const CROSS_TENANT_ROLES = new Set([SystemRole.SUPER_ADMIN, SystemRole.ADMIN]);

if (!CROSS_TENANT_ROLES.has(callerRoleName) && callerClinicId !== id) {
  throw new ForbiddenException('Solo puede modificar su propia sede');
}
```

2.4 Pruebas agregadas

- `tenant.interceptor.spec.ts` — 4 pruebas: reproduce el contexto GraphQL real, confirma que el bug original existía (regresión), confirma el comportamiento correcto en HTTP, y valida `clinicId` nulo para usuarios cross-tenant.
- `update-clinic.use-case.spec.ts` — 4 pruebas: deniega a rol acotado modificar otra sede, permite modificar la propia, permite a rol de plataforma modificar cualquiera, y NotFound antes de evaluar el tenant.

3. A02 — Fallas criptográficas (Cryptographic Failures)

El servicio centralizado de hashing de contraseñas (`PasswordService` , único punto de hashing en tiempo de ejecución del sistema) usaba un *cost factor* de bcrypt de `10` . Se elevó a `12` , dentro del rango recomendado por OWASP, para dar margen frente al avance de hardware sin impacto perceptible en el tiempo de login.

```
// server/src/modules/auth/infrastructure/services/password.service.ts
private readonly SALT_ROUNDS = 12; // antes: 10
```

Prueba agregada: se verifica que el cost factor real quede codificado en el hash resultante (prefijo `$2b$12$`) y se confirma el round-trip `hash` / `compare`. Los scripts de seed de datos de desarrollo (`prisma/seed.ts`), que usan una contraseña de prueba fija, se dejaron fuera de este cambio por no ser superficie de ataque real.

4. A06 — Componentes vulnerables y desactualizados

Se agregó un job `audit` al pipeline de CI (`.github/workflows/ci.yml`) que corre `pnpm audit --prod --audit-level high` sobre ambos workspaces (`server` y `client`) en cada push/PR a `main`.

Por qué quedó en modo informativo (`continue-on-error`): al ejecutar la auditoría se detectaron ~20 vulnerabilidades *high/critical* preexistentes en dependencias de producción (mayormente transitivas, arrastradas por Express, Prisma y Apollo — p.ej. `multer`, `hono`, `handlebars`, `ws`). Activar el gate como bloqueante hoy habría dejado **todos** los PRs en rojo de forma inmediata. Se prioriza visibilidad continua sobre bloqueo prematuro; el gate debe subirse a obligatorio una vez saneada la deuda.

Pendiente: evaluar actualización de `handlebars` (parche, bajo riesgo) y `nodemailer` (cambio mayor de versión, requiere revisión de API) como dependencias directas; el resto requiere `overrides` de pnpm con riesgo de incompatibilidad, a evaluar caso por caso.

5. A09 — Fallas de registro y monitoreo de seguridad

El sistema no dejaba rastro de eventos de seguridad relevantes (intentos de login fallidos, accesos denegados por permisos). Se implementó una bitácora de auditoría dedicada:

- **Tabla `SecurityAuditLogs`** (append-only, sin claves foráneas de forma deliberada: el registro debe sobrevivir el borrado del usuario o la clínica que lo originó; `userId` / `email` / `clinicId` se guardan como snapshot del momento del evento).
- **`SecurityAuditService`**: registra eventos de forma *fire-and-forget* — un fallo al escribir la auditoría nunca convierte una respuesta 401/403 en un error 500 ni bloquea la petición del usuario.
- **Eventos cubiertos:** `LOGIN_FAILED` (capturado en `AuthController` al fallar la autenticación) y `PERMISSION_DENIED` (capturado en `PermissionsGuard` al denegar acceso por falta de permisos).

El alcance se limitó deliberadamente a estos dos eventos para evitar ruido y mantener foco; quedó fuera de esta ronda un endpoint de consulta/monitoreo de la bitácora (hoy solo se escribe, no hay vista de lectura para el equipo de operaciones).

6. Categorías no auditadas en esta ronda

Las siguientes categorías no fueron objeto de revisión dirigida durante este trabajo. Se listan las mitigaciones estructurales ya presentes en el stack del proyecto, sin que constituyan una auditoría formal.

- **A03 — Inyección:** el uso exclusivo de Prisma ORM (consultas parametrizadas) y `class-validator` en los DTOs del servidor reduce la superficie de inyección SQL/NoSQL clásica, pero no se auditó validación de entrada caso por caso ni inyección en otros vectores (logs, comandos del sistema, plantillas de correo).

- **A04 — Diseño inseguro:** sin revisión de modelado de amenazas formal.
- **A05 — Configuración insegura:** sin auditoría de headers HTTP, CORS, variables de entorno expuestas, ni configuración de Docker/infraestructura.
- **A07 — Fallas de identificación y autenticación:** el sistema ya usa JWT + Passport con rotación de tokens; la mejora de A02 (bcrypt) aporta parcialmente a esta categoría, pero no se revisaron políticas de bloqueo de cuenta, fuerza de contraseña, ni expiración de sesión.
- **A08 — Fallas de integridad de software y datos:** sin revisión de integridad de pipeline de CI/CD ni de deserialización de datos no confiables.
- **A10 — Server-Side Request Forgery (SSRF):** sin auditoría de llamadas salientes del servidor (p.ej. integración con MercadoPago, envío de correo) que reciban URLs o destinos influenciados por el usuario.

7. Commits y trazabilidad

Commit	Categoría	Descripción
e59d67f	A01	Tenant scoping en GraphQL (no solo REST)
cdbcd4b	A09	Bitácora de auditoría de eventos de seguridad
b9a8f95	A02	Cost factor de bcrypt de 10 a 12
247b27f	A06	Auditoría de dependencias en CI
5ecc615	A01	Guard de tenant en updateClinic (defense-in-depth)
90d25c5	—	Limpieza de lint introducido por los cambios de seguridad
e754494	—	Merge a <code>main</code> vía PR #35

8. Verificación

- Suite de pruebas completa del backend: **39 suites / 288 pruebas**, todas en verde tras la integración de todos los cambios.
- `tsc --noEmit` : 0 errores.
- Migración de base de datos de `SecurityAuditLogs` aplicada y verificada mediante inserción/borrado de prueba en la base de datos real.
- 13 pruebas unitarias nuevas dedicadas a los hallazgos de seguridad de esta ronda (tenant interceptor, permissions guard, password service, update-clinic).

9. Hallazgos pendientes y deuda conocida

- **A06:** ~20 vulnerabilidades high/critical en dependencias transitivas de producción, sin sanear; el gate de CI permanece informativo hasta resolverlas.
- **A09:** falta endpoint de consulta de la bitácora para el equipo de operaciones (hoy solo escritura).
- **Deuda de calidad de código (no es un hallazgo OWASP):** el pipeline de CI tiene el paso de *lint* en rojo de forma crónica (~1740 problemas preexistentes de `@typescript-eslint/no-unsafe-*` en todo el código base, no introducidos por este trabajo). Se corrigieron los puntos introducidos por los cambios de esta ronda; sanear el resto es un esfuerzo independiente.

- **A03, A04, A05, A07, A08, A10:** pendientes de una ronda de auditoría dirigida.

Informe generado a partir del trabajo de remediación de seguridad realizado sobre la rama `feat/payments-mp-redirect`, integrada a `main` el 30 de junio de 2026 (PR #35, commit de merge `e754494`). Referencia: OWASP Top 10:2021 — owasp.org/Top10.